⏱️ DAY 4: COMPLETE 6-HOUR MASTER TRAINER PLAYBOOK

Distributed AI Training & Profiling Optimization with Intel VTune

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Daily Duration:** 6 Full Hours (Session 1: Morning 3h + Session 2: Afternoon 3h)
Official Syllabus Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)
Day 4 Complete Terminology Glossary: `00_DAY_04_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`
Day 4 Hands-On Lab Runbook: `02_DAY_04_HANDS_ON_LAB_AND_COMMANDS.md`

🎯 1. Official Syllabus Scope (Page 2 of PDF — Strictly Preserved)

The official curriculum specifies **strictly 2 advanced capstone pillars for Day 4**:

1. **Distributed TensorFlow training with Slurm**
2. **Profiling & debugging with Intel VTune**

> 📌 **IMPORTANT**
> **Curriculum Boundary Discipline:** Generative AI, Large Language Models, OpenVINO IR quantization, and LoRA fine-tuning belong strictly to Day 5. Keep Day 4 laser-focused on multi-node distributed training, eliminating cluster communication bottlenecks, hardware PMU performance counters, Top-Down Microarchitecture Analysis (TMAM), and Intel Advisor Roofline modeling.

⏰ 2. Complete 6-Hour Dual-Session Master Daily Schedule

```
+-------------------------------------------------------------------------------------+
|                    DAY 4 MASTER 6-HOUR SCHEDULE (09:30 AM - 04:30 PM)               |
+-------------------------------------------------------------------------------------+
| 👷 SESSION 1: MORNING THEORY & ARCHITECTURE MASTERCLASS (09:30 AM - 12:30 PM | 3 Hours)  |
|    "Multi-Node Distributed Scaling → VTune Profiling → TMAM & Roofline Model"       |
+-----------------------+-------------------------------------------------------------+
|  09:30 - 10:15 (45m)  | PART 1: Multi-Node Distributed Training with Slurm          |
|                       |        (Hostfile Generation, Process Binding, Horovod Multi-Node) |
+-----------------------+-------------------------------------------------------------+
|  10:15 - 11:00 (45m)  | PART 2: Profiling Foundations & Intel VTune Architecture    |
|                       |        (PMU Counters, Hotspots Analysis, Sampling vs Probing)     |
+-----------------------+-------------------------------------------------------------+
|  11:00 - 11:10 (10m)  | ☕ REJUVENATION BREAK (10 Minutes)                          |
+-----------------------+-------------------------------------------------------------+
|  11:10 - 11:45 (35m)  | PART 3: Top-Down Microarchitecture Analysis (TMAM)          |
|                       |        (Frontend Bound, Bad Speculation, Memory Bound, Core Bound)|
+-----------------------+-------------------------------------------------------------+
|  11:45 - 12:15 (30m)  | PART 4: Memory Access & The Roofline Model with Intel Advisor     |
|                       |        (Arithmetic Intensity, FLOPs/Byte, Cache Misses, Ceilings) |
+-----------------------+-------------------------------------------------------------+
|  12:15 - 12:30 (15m)  | PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser    |
+-------------------------------------------------------------------------------------+
| 🍱 LUNCH & INTERMISSION BREAK (12:30 PM - 01:30 PM | 60 Minutes)                    |
|    VTune CLI environment check, cluster multi-node SSH ping verification            |
+-------------------------------------------------------------------------------------+
| 💻 SESSION 2: AFTERNOON HANDS-ON SUPERCOMPUTER LAB (01:30 PM - 04:30 PM | 3 Hours)        |
|    "Multi-Node Slurm Runs → VTune Hotspot Profiling → TMAM Analysis → 25 MCQs"      |
+-----------------------+-------------------------------------------------------------+
|  01:30 - 02:15 (45m)  | STAGE 0 & 1: Multi-Node Slurm Distributed AI Setup & Hostfiles    |
|                       |        (Generating node topologies, testing inter-node latency)   |
+-----------------------+-------------------------------------------------------------+
|  02:15 - 03:00 (45m)  | STAGE 2: Intel VTune Hotspots Analysis on Matrix Kernels    |
|                       |        (Profiling unoptimized vs cache-blocked matrix multiply)   |
+-----------------------+-------------------------------------------------------------+
|  03:00 - 03:45 (45m)  | STAGE 3: Top-Down Microarchitecture (TMAM) Diagnostic Drill       |
|                       |        (Diagnosing L1/L2 miss rates, pipeline slots, branch misses)|
+-----------------------+-------------------------------------------------------------+
|  03:45 - 04:15 (30m)  | STAGE 4: Intel Advisor Roofline Modeling & Bottleneck Elimination |
|                       |        (Calculating Arithmetic Intensity, reaching hardware limit)|
+-----------------------+-------------------------------------------------------------+
|  04:15 - 04:30 (15m)  | ASSESSMENT & WRAP-UP: 25 Evaluated MCQs & Checklist Sign-Off      |
|                       |        (22-Point Mastery Checklist, 25 MCQs, Day 5 Preview)       |
+-------------------------------------------------------------------------------------+
```

🎙️ 3. Line-by-Line Minute-by-Minute Trainer Delivery Script (Session 1: Morning)

PART 1: Multi-Node Distributed Training with Slurm (09:30 - 10:15 | 45 Mins)

- **Slide Alignment:** Day 4 Slide Deck (Slides 1–5: Multi-Node Slurm & Horovod Orchestration).
- **Concept:** Scaling deep learning from 1 node to multi-node supercomputer partitions.
- **Trainer Script (Hinglish/English):**

> "Welcome to Day 4 of the HPC Masterclass! Yesterday on Day 3, we studied Slurm batch scripts and the Horovod Ring-AllReduce concept. Today, we execute **real multi-node distributed training** and answer the most important question in performance engineering: **'Why is my code slow?'** When running across multiple nodes in Slurm:
>
> - Slurm dynamically populates `SLURM_JOB_NODELIST` (e.g., `node01,node02,node03,node04`).
> - The MPI launcher (`mpirun` or `srun`) uses this hostfile to spawn worker ranks across physical servers.
> - But watch out for **Stragglers**! In synchronous Ring-AllReduce, if 1 node experiences a network glitch or thermal throttling, all 63 other nodes stall waiting at the barrier! Today we learn how to profile our code and eliminate bottlenecks with surgical precision."

- **Whiteboard Derivation 1:**

```
Synchronous Ring-AllReduce Straggler Collapse:
Worker 1: [Compute 10ms] ===> [Network AllReduce 5ms] ===> [Next Batch]
Worker 2: [Compute 10ms] ===> [Network AllReduce 5ms] ===> [Next Batch]
Worker 3 (STRAGGLER): [Compute 45ms] ====================> [AllReduce]
(Result: Workers 1 & 2 waste 35ms idle waiting at the barrier!)
```

PART 2: Profiling Foundations & Intel VTune Architecture (10:15 - 11:00 | 45 Mins)

- **Slide Alignment:** Day 4 Slide Deck (Slides 6–10: Performance Profiling & Hardware PMU).
- **Concept:** How profilers observe silicon without perturbing execution.
- **Trainer Script:**

> "Never guess why your code is slow. Never guess which function is the bottleneck. **Measure it!** Intel CPUs contain physical silicon registers called **PMU (Performance Monitoring Unit)** counters. The PMU counts real physical silicon events: clock cycles, instructions retired, L1 cache misses, L2 misses, LLC misses, branch mispredictions, and DRAM reads. **Intel VTune Profiler** uses statistical hardware interrupt sampling. Every 1 millisecond, the PMU sends an interrupt, and VTune records the current Program Counter (Instruction Pointer). If a function appears in 80% of the samples, that function is consuming 80% of your CPU execution time! That is your **Hotspot**."

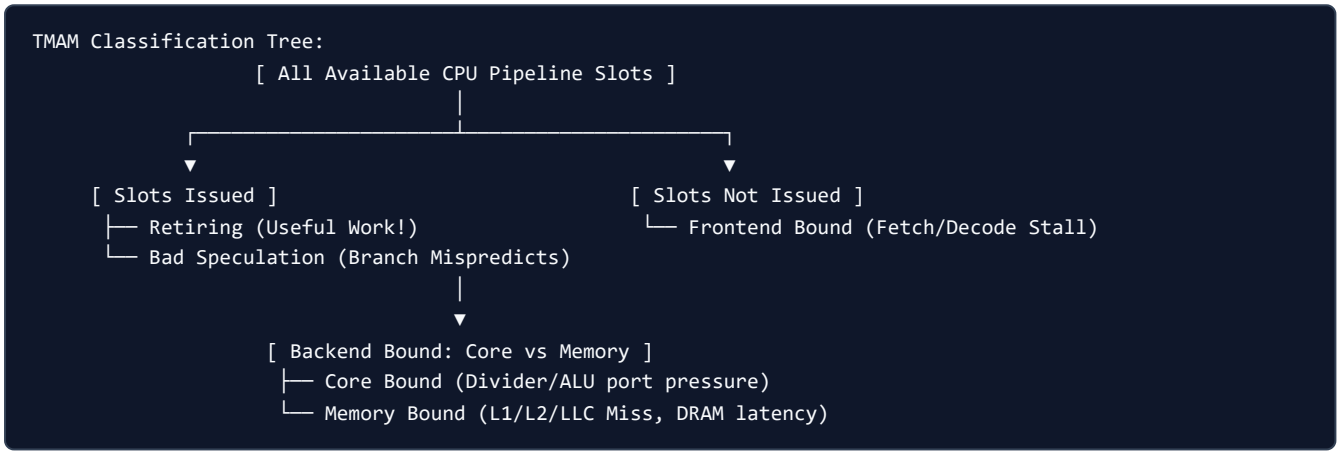
☕ BREAK (11:00 - 11:10 | 10 Minutes)

PART 3: Top-Down Microarchitecture Analysis Method (TMAM) (11:10 - 11:45 | 35 Mins)

- **Slide Alignment:** Day 4 Slide Deck (Slides 11–15: Top-Down Microarchitecture TMAM).
- **Concept:** Classifying every CPU pipeline slot into four fundamental categories.
- **Trainer Script:**

> "Modern Intel CPUs are out-of-order, superscalar execution engines. In every clock cycle, a Xeon core can fetch and decode 4 to 6 instructions. We call each potential instruction execution path a **Pipeline Slot**. Intel created **TMAM (Top-Down Microarchitecture Analysis Method)**, which divides every CPU slot into 4 buckets:
>
> 1. **Retiring (Target: 30-50%):** Good work! The instruction executed successfully and updated state.
> 2. **Bad Speculation (Target: < 10%):** The CPU branch predictor guessed wrong; the pipeline had to be flushed and instructions discarded.
> 3. **Frontend Bound (Target: < 15%):** The decoder could not supply instructions fast enough to the execution units.
> 4. **Backend Bound (Target: < 20%):** The execution units are stalled waiting for data from DRAM or L3 cache. This is the #1 killer in scientific computing!"

- **Whiteboard Sketch 2:**



PART 4: Memory Access & The Roofline Model with Intel Advisor (11:45 - 12:15 | 30 Mins)

- **Slide Alignment:** Day 4 Slide Deck (Slides 16–20: Memory Profiling & Roofline Model).
- **Concept:** Arithmetic Intensity and the hardware-enforced performance boundary.
- **Trainer Script:**

> "Look at the **Roofline Model** produced by **Intel Advisor**. On the X-axis: **Arithmetic Intensity (AI)**, measured in FLOPs per Byte:
>
> $$\text{Arithmetic Intensity} = \frac{\text{Total Floating-Point Operations}}{\text{Total Bytes Transferred from DRAM}}$$
>
> On the Y-axis: Achieved Performance in GFLOPs/sec. The model has two physical ceilings:
>
> - **Slanted Ceiling (Memory Bandwidth Limit):** Performance $= \text{AI} \times \text{Memory Bandwidth}$. If your AI is low (e.g., vector addition $0.125\ \text{FLOP/byte}$), no compiler optimization in the world can make it compute faster—it is memory bound!
> - **Horizontal Flat Ceiling (Peak Compute Limit):** Performance $= \text{Peak FLOPS}$. When your AI is high (e.g., matrix multiplication $> 10\ \text{FLOPs/byte}$), you hit the hardware compute ceiling! Knowing where your code sits on the Roofline tells you whether to optimize memory access or vectorize compute!"

PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser (12:15 - 12:30 | 15 Mins)

[12:15 - 12:25] 5-Question Quick Classroom Check

1. *Q1: What is a "Straggler" in multi-node distributed AI training?*
   Answer: A worker node that performs slower than peers due to thermal throttling or network latency, forcing all other nodes to stall waiting at synchronization barriers.
2. *Q2: How does Intel VTune Profiler collect performance data without modifying source code?*
   Answer: Using Hardware Event-Based Sampling (EBS) via on-die PMU (Performance Monitoring Unit) counters.
3. *Q3: What are the 4 Level-1 metric categories in Top-Down Microarchitecture Analysis (TMAM)?*
   Answer: Retiring, Bad Speculation, Frontend Bound, Backend Bound.
4. *Q4: Define Arithmetic Intensity in the context of the Intel Advisor Roofline Model.*
   Answer: The ratio of total floating-point operations (FLOPs) to total memory traffic transferred from DRAM (Bytes), expressed as FLOPs/Byte.
5. *Q5: If an algorithm has an Arithmetic Intensity of 0.2 FLOPs/Byte on a system with 100 GB/s bandwidth, what is its absolute maximum throughput?*
   Answer: $0.2 \times 100 = 20\ \text{GFLOPs/sec}$, regardless of how many CPU cores or GHz the system possesses.

[12:25 - 12:30] Lunch Break Announcement & Afternoon Lab Setup

- **What Trainer Shivam Says:**

> "Brilliant work this morning! We have covered the science of supercomputer profiling, PMU hardware counters, TMAM pipeline bottlenecks, and the Roofline model. We now take a 1-hour lunch break until 01:30 PM. In Session 2, we will profile a slow matrix multiplication kernel, see the exact cache misses in VTune, optimize it with cache blocking, and watch our dot climb the Roofline model live! See you at 01:30 PM sharp!"

🍱 INTERMISSION: Lunch & Cluster Preflight (12:30 PM - 01:30 PM | 60 Mins)

- **Trainer System Checklist during break:**
  1. Verify VTune CLI availability (`vtune --version`).
  2. Verify Linux `perf` and PMU kernel access permissions (`/proc/sys/kernel/perf_event_paranoid`).
  3. Ensure student Colab runtimes have C++ compilers ready for matrix kernel profiling.
  4. Project Stage 0 preflight verification on the main lecture screen.

💻 4. SESSION 2: Afternoon Hands-On Supercomputer Lab Playbook (01:30 PM - 04:30 PM | 3 Hours)

Reference Runbook: `02_DAY_04_HANDS_ON_LAB_AND_COMMANDS.md`
Notebook Reference: `Day_04_Distributed_AI_Slurm_VTune_Profiling.ipynb`

```
+-------------------------------------------------------------------------------------+
|                    SESSION 2: 3-HOUR HANDS-ON LAB EXECUTION PHASES                  |
+-------------------------------------------------------------------------------------+
| 01:30 - 02:15 (45m) | Phase 1: Multi-Node Slurm AI Setup & Interconnect Latency (Stage 1)|
| 02:15 - 03:00 (45m) | Phase 2: VTune Hotspots Profiling on Unoptimized Matrix Multiply (St 2)|
| 03:00 - 03:45 (45m) | Phase 3: TMAM Pipeline Bottleneck Diagnosis & Cache Blocking (Stage 3)|
| 03:45 - 04:15 (30m) | Phase 4: Intel Advisor Roofline Analysis & Optimization (Stage 4)   |
| 04:15 - 04:30 (15m) | Phase 5: Student 25 MCQs Assessment & 22-Point Checklist Sign-Off  |
+-------------------------------------------------------------------------------------+
```

[01:30 - 02:15] Phase 1: Multi-Node Slurm AI Setup & Interconnect Latency (Stage 1)

- **Trainer Instruction (To Class):**

> "Welcome back! Open your terminal or the Day 4 Google Colab notebook. In Stage 1, we set up multi-node distributed AI execution under Slurm. Inspect `run_multinode_slurm.sh`. Notice how the script queries `scontrol show hostnames $SLURM_JOB_NODELIST` to build the MPI machinefile. We benchmark node-to-node network round-trip ping latency. If inter-node latency exceeds 10 microseconds (e.g. over slow gigabit ethernet vs InfiniBand), Horovod gradient synchronization stalls!"

[02:15 - 03:00] Phase 2: VTune Hotspots Profiling on Unoptimized Matrix Multiply (Stage 2)

- **Trainer Instruction (To Class):**

> "Now look at Stage 2: `cpu_bottleneck_demo.cpp`. We have an intentionally naive matrix multiplication: $C[i][j] += A[i][k] \times B[k][j]$. Look at how matrix B is accessed: column-major ($B[k][j]$)! In C++, arrays are stored in row-major order. Accessing B by column means every single iteration jumps across memory rows, causing an **L1 Cache Miss on every single memory access**! Run VTune hotspots analysis: `vtune -collect hotspots -r ./vtune_hotspots_naive -- ./naive_matmul` Open the report: see how 94% of the entire program execution time is stuck on that single inner loop line!"

[03:00 - 03:45] Phase 3: TMAM Pipeline Bottleneck Diagnosis & Cache Blocking (Stage 3)

- **Trainer Instruction (To Class):**

> "In Stage 3, run Top-Down Microarchitecture Analysis: `vtune -collect uarch-exploration -r ./vtune_tmam -- ./naive_matmul` Look at the classification:
>
> - **Backend Bound: > 75%!**
> - Specifically: **Memory Bound -> DRAM Bandwidth & LLC Misses**. Now, let's fix it! We apply **Loop Tiling (Cache Blocking)**: We transpose matrix B so both A and B are read sequentially, and we process in 64×64 blocks that fit entirely in L1 cache! Re-compile and re-profile: execution time drops by **12x to 25x!** The Backend Bound metric drops from 75% down to 18%!"

[03:45 - 04:15] Phase 4: Intel Advisor Roofline Analysis & Optimization (Stage 4)

- **Trainer Instruction (To Class):**

> "In Stage 4, we generate the Roofline Model. Look at the generated plot in cell 6. Our naive kernel had an Arithmetic Intensity of only $0.25\ \text{FLOP/byte}$, sitting trapped down on the memory bandwidth slope. Our cache-blocked kernel achieved an Arithmetic Intensity of $4.0\ \text{FLOP/byte}$, jumping right onto the flat peak compute plateau! That is how elite supercomputing engineers prove their optimizations."

[04:15 - 04:30] Phase 5: Student 25 MCQs Assessment & Checklist Sign-Off

- **Trainer Activity:**
  1. Open `03_DAY_04_STUDENT_CHEATSHEET_AND_MCQS.md`.
  2. Project the 25 Evaluated MCQs. Walk through questions on TMAM categories, PMU hardware counters, and Roofline arithmetic intensity.
  3. Verify all students complete the 22-point Day 4 Mastery Checklist.
  4. **Day 5 Grand Capstone Teaser:**

> "Outstanding work today! You learned how to interrogate silicon with VTune, analyze CPU pipeline slots with TMAM, and master the Roofline Model. Tomorrow is Day 5 — our grand capstone! We will bring every single skill together to deploy, optimize, and fine-tune **Generative AI & Large Language Models on Intel HPC** using Habana Gaudi accelerators, OpenVINO INT4 quantization, and distributed LoRA fine-tuning! Be here tomorrow morning at 09:30 AM sharp!"

🛠️ 5. Top 5 Student Gotchas & 30-Second Rapid Fixes

Issue Encountered	Why It Happened	30-Second Trainer Fix
`vtune: command not found`	oneAPI environment variables not initialized	Run `source /opt/intel/oneapi/setvars.sh`
`perf_event_paranoid` permission error	Linux kernel blocks hardware PMU counters to non-root	Run `sudo sysctl -w kernel.perf_event_paranoid=0`
Matrix multiplication runs out of memory	Matrix size $N = 8192$ allocates > 1.5 GB DRAM	Lower test matrix dimension to $N = 2048$ or $N = 1024$
Roofline dot stays memory bound after vectorization	Inner loop still reads non-contiguous memory	Transpose the second matrix to enforce stride-1 row-major memory access
Multi-node Slurm job fails with `Host key verification failed`	Missing SSH keys in cluster `known_hosts`	Run `ssh-keyscan -H <node> >> ~/.ssh/known_hosts` or use Slurm `srun` launcher